

# A Guarded Single-Core Architecture for Agreement-Forest Heuristics

PACE 2026 Heuristic-Track Solver Architecture

June 2026

## Abstract

This paper describes a final simplified architecture for a heuristic solver for the PACE 2026 agreement-forest setting. The architecture is built around a small number of implementation concepts: tree annotations, exact common-subtree reduction, validated component candidates, weighted set packing, and one guarded routing rule. Instances with at most 2200 leaves use a score-sensitive small/medium kernel followed by a unified repair acceptance stage. Larger instances use a single candidate-pool core that treats valid components as weighted sets and packs a compatible subset under atom and edge conflict constraints. The design is intentionally not a portfolio: it does not race many routes, use cached public-instance answers, or select behavior by instance name. Metafeatures are still used, but only as local budget and refinement signals inside the kernels rather than as a highly coupled top-level route table.

## 1 PACE 2026 Heuristic Track Problem

The PACE 2026 announcement frames the scientific topic as *agreement forests*. A phylogenetic tree is a rooted directed tree whose leaves are in one-to-one correspondence with the taxa labels. An agreement forest for input trees  $T_1, \dots, T_t$  is a forest that can be obtained from each input tree by deleting directed edges and then cleaning up the resulting degree-one unlabeled vertices. The size of the forest is the number of resulting phylogenetic trees. A forest of minimum size is called a maximum-agreement forest because it preserves as much common structure as possible.

The solver described here targets the heuristic track, where each instance has two rooted phylogenetic trees over the same leaf-label set. The task is not to prove optimality, but to return a valid agreement forest with as few components as possible within the contest time budget. The announcement also highlights two qualitatively different data regimes: real-world data is expected to have few trees, small agreement-forest size, and many leaves, while generated data may contain large agreement-forest sizes and many leaves. This split is one of the reasons the architecture below treats small/medium score-sensitive cases differently from large candidate-packing cases.

Figure 1 shows a small example. The two input trees have the same leaves  $\{1, 2, 3, 4\}$ , but their whole-tree topologies differ. Cutting the highlighted edge to leaf 4 in each input and then cleaning up unary vertices leaves the same rooted component on  $\{1, 2, 3\}$ : leaves 1 and 2 form a cherry, and leaf 3 attaches above that cherry. Leaf 4 becomes a singleton component. Thus  $\mathcal{F} = \{\{1, 2, 3\}, \{4\}\}$  is an agreement forest of size two. The all-singleton partition would also be valid, but it would have size four and therefore much lower quality.

## 2 Problem Setting and Design Goals

The solver receives two rooted phylogenetic trees with the same leaf labels and must output a partition of the labels into valid components. The component count is the objective: fewer

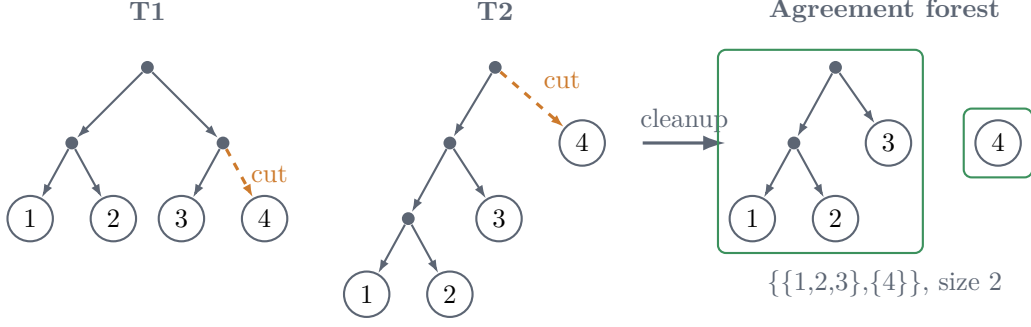


Figure 1: A four-leaf agreement-forest example for the heuristic-track setting. Both input trees can be cut and cleaned up to produce the same two components.

components are better. Let  $k^*$  be the reference component count for an instance with  $n$  leaves. The score used in the local evaluation is

$$1 - \frac{\max(0, k - k^*)}{\max(1, n - k^*)},$$

where  $k$  is the solver’s output component count. This formula makes small denominators important: a one-component regression on a near-tight small instance can hurt more than many components on a huge instance.

### 3 Architecture Invariants

The implementation is organized around three invariants that are preserved across all branches. First, every output must be a partition of the original label set: labels appear exactly once, and every reduced atom expands through `atom_to_original_mask` before rendering. If packing covers only part of the reduced atom universe, the uncovered atoms become singleton components during output assembly, so the rendered forest is still complete.

Second, component validity is centralized. Candidate generators may be broad, local, or repair-driven, but they do not decide correctness on their own. Multi-atom candidates are accepted only after their restricted signatures match in both trees and their tree-edge resources have been recorded. This invariant keeps pair candidates, interval candidates, current-component unions, and blocker-region repairs under the same validity contract.

Third, the incumbent is monotone in component count. Search, packing, and repair stages are allowed to propose replacements, but an accepted replacement must render as a valid partition and must not increase the component count. This lets later stages be opportunistic: an unaccepted proposal simply leaves the incumbent unchanged.

*Main implementation components.* The partition invariant is enforced by `atom_masks_to_components`, `selected_masks_to_components`, `render_output`, and `validate_components`. Candidate validity is concentrated in `CandidatePool.validate_mask` and `CandidatePool.add_mask_weighted`. Monotone acceptance is implemented by the repair wrappers that compare output counts before replacing an incumbent. Incumbent state is stored through `set_current_components` and `current_output`.

### 4 High-Level Architecture

The fixed architecture is called the *guarded single-core* architecture. It is intentionally expressed as one high-level rule:

$$n \leq 2200 \Rightarrow \text{score-sensitive small/medium kernel}, \quad n > 2200 \Rightarrow \text{single candidate-pool core}.$$

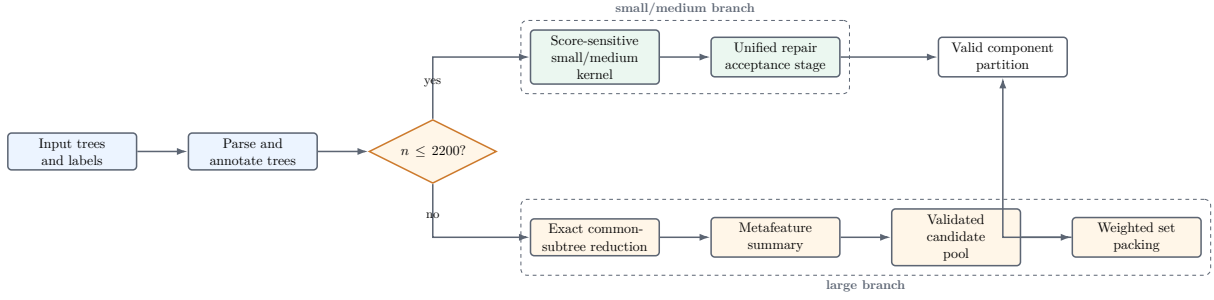


Figure 2: Pipeline stages of the guarded single-core solver. The architecture has one policy decision, then either a score-sensitive small/medium route or a single large-instance candidate-packing route.

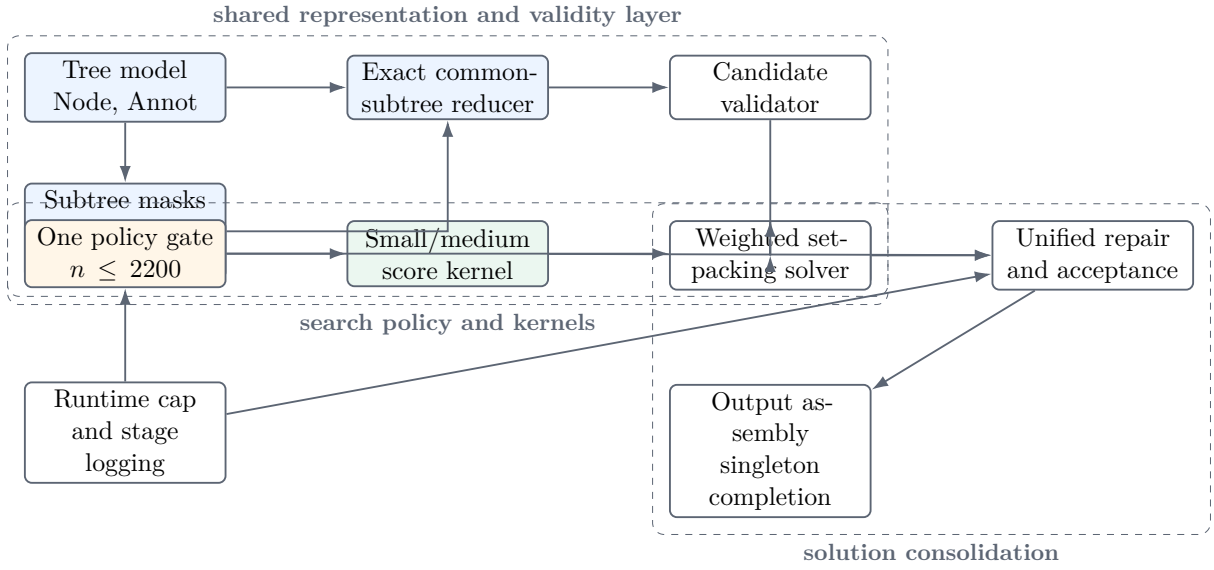


Figure 3: Main components and dependency direction. The guard chooses between two search kernels, but both use the same representation, candidate validity checks, packing concepts, and output assembly.

The rule is not a portfolio in the usual sense. It does not race many solvers, does not select from a catalog of instance-name patterns, and does not run a large collection of postprocessors. It simply protects the cases where the single core was empirically the wrong abstraction.

*Main implementation components.* The top-level rule is implemented by `_solve` and `_exec_simple_guarded_core`. The latter reads `PACE26_SIMPLE_GUARDED_SMALL_MAX_N`, emits the `guarded_policy_gate` stage through `_stage_instant`, and then dispatches either to `_exec_score_sensitive_small` or `_exec_single_core`. Stage timing and before/after component counts are recorded through `_stage_run_text`.

## 5 Main Components

Figure 3 shows the main implementation components and their dependencies. The central idea is to keep the data model and validation layer shared, while separating only the two search regimes that have different score economics.

## 6 Essential Data Structures

The implementation is built around a small set of explicit data structures. They are deliberately simple: most expensive structural relations are encoded as integer bit masks, and most correctness checks flow through one validation cache.

| Structure              | Role in the architecture                                                                                                                                                                                                                                                                                                   |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Node</b>            | Rooted tree node. It stores <b>children</b> , a one-bit <b>label_mask</b> for leaves or an atom mask for contracted leaves, a <b>parent</b> , a stable <b>id</b> , and a <b>parent_edge_id</b> . The edge id is what allows later candidate validation to represent tree-edge usage as bit masks.                          |
| <b>Annot</b>           | Per-node annotation containing <b>subtree_mask</b> , canonical <b>signature</b> , and <b>leaf_count</b> . The signature is recursive: leaves are labeled signatures and internal nodes are sorted tuples of child signatures. This makes exact common-subtree detection and restricted-component validation deterministic. |
| <b>ReducedInstance</b> | The reduced pair of trees plus <b>atom_to_original_mask</b> . Each reduced leaf atom expands back to one or more original labels, so the solver can search on atoms and render the final answer on original labels.                                                                                                        |
| <b>Candidate</b>       | A validated potential component. It stores <b>atom_mask</b> , packing <b>weight</b> , two edge masks, two component-root masks, and <b>size</b> . These fields are exactly the quantities needed by weighted set packing and local repair.                                                                                 |
| <b>ValidationInfo</b>  | Intermediate validation record for a mask: restricted signature, edge masks, root masks, and size. It prevents candidate generation from duplicating correctness logic.                                                                                                                                                    |
| <b>CandidatePool</b>   | Shared search state for the large branch: reduced trees, annotations, leaf arrays, depth maps, leaf-order ranks, candidate list, <b>by_mask</b> deduplication, validation cache, optional conflict bitsets, and mode flags for direct repair or conservative edge handling.                                                |
| <b>InstanceMeta</b>    | Small set of metafeatures: atom ratio, cluster Jaccard, sampled soft-cherry agreement, sampled triplet agreement, and a hard-edge bucket flag. These features tune local repair behavior without becoming a top-level instance-name route table.                                                                           |

Table 1: Core data structures used by the guarded architecture.

**Bit masks are the central representation.** A component, subtree, or candidate is represented by an integer mask. Intersections such as `candidate.atom_mask & used_atoms` test whether two candidates cover the same atom. Similar masks over edge ids test whether two candidates reuse the same tree resource. This is the reason the packing loop can remain a compact greedy heuristic instead of requiring an external optimization model.

**Validation is cached at the mask level.** Candidate generators may propose the same mask through different paths: pairs, order windows, seed unions, local repair neighborhoods, or blocker regions. The pool’s `validation_cache` maps each mask to either a `ValidationInfo` record or a cached invalid result. This prevents the same restricted-signature and edge-collection work from being repeated.

## 7 Pipeline Stages and Implementation Details

### 7.1 Tree Representation and Annotations

The representation layer stores the two input trees as rooted node structures with stable labels. Each internal node receives annotations: subtree mask, signature, leaf count, and traversal metadata. This layer is foundational because every later step asks the same basic question: whether a proposed set of labels induces compatible restricted structures in both trees.

The use of bit masks is deliberate. It gives constant-time disjointness tests, compact candidate identities, and cheap conflict checks. It also keeps the core loops close to the combinatorial structure of the agreement-forest problem.

Implementation begins by assigning stable node ids and parent-edge ids on both trees. The ids index annotation dictionaries, while parent-edge ids give a compact namespace for resource-conflict masks. Annotation is a postorder computation. A leaf receives signature ( $L, \text{label\_mask}$ ). An internal node receives the union of its child subtree masks, the sum of child leaf counts, and a canonical signature ( $I, \text{sorted child signatures}$ ). Because child signatures are sorted, the signature is insensitive to child ordering and can be compared directly across the two trees.

The large branch also constructs leaf arrays, depth maps, and leaf-order ranks. These support fast LCA-style operations for pairs, triplet prechecks, and rank-based component-root lookup. For two-leaf masks, validation can use a special pair path routine that directly returns the used edge masks. For larger masks, it finds the component root and computes the restricted signature and edge set from the annotation dictionaries.

*Main implementation components.* Parsing and representation are carried by `read_instance_from_text`, `parse_newick`, and the `Node` dataclass. Stable structural ids come from `assign_ids_and_edges`; traversal helpers such as `walk_nodes`, `collect_leaves`, and `leaf_nodes_and_depths` build the node arrays used later. `compute_annotations` creates the `Annot` records, and `leaf_order_positions` supplies ranks for fast component-root lookup.

### 7.2 Exact Common-Subtree Reduction

Exact common-subtree reduction contracts identical subtrees in both input trees into atoms before search. This is a safe reduction: it preserves the structure needed by the agreement-forest objective while shrinking the effective problem. It is the highest-value preprocessing stage because it improves both quality and speed without introducing heuristic risk.

The reduction also helps the simplified large branch. A single candidate-pool core has less late repair capacity than the full architecture, so giving it a smaller and cleaner atom-level instance is more important.

The implementation repeatedly annotates both current trees, builds maps from internal subtree masks to nodes, and searches for masks that occur in both trees with identical canonical signatures. The full-tree mask is ignored, because contracting it would erase the problem. Every accepted common mask is contracted to a reduced leaf atom in both trees. The `atom_to_original_mask` array records which original labels belong to that atom. The process repeats until no more exact common masks remain or the runtime guard expires.

This stage is safe in a stronger sense than ordinary heuristic compression: the same labeled rooted subtree appears on both sides, so it can be treated as an indivisible component for later search and expanded back during output assembly. For example, on labels  $\{1, 2, 3, 4, 5\}$ , suppose

$$T_1 = ((1, 2), (3, (4, 5))), \quad T_2 = ((1, 2), ((4, 5), 3)).$$

Both trees contain the identical rooted subtrees  $(1, 2)$  and  $(4, 5)$ . The reducer can contract them into atoms

$$a = \{1, 2\}, \quad b = \{4, 5\}, \quad c = \{3\}.$$

If a later stage returns reduced components  $\{a, c\}$  and  $\{b\}$ , output assembly expands them to original-label components  $\{1, 2, 3\}$  and  $\{4, 5\}$ . In the h26 trace, this same mechanism reduces 14734 original leaves to 12206 atoms in 1.826 seconds before the packing stage begins.

*Main implementation components.* This stage is realized by `reduce_exact_common_subtrees`. It repeatedly calls `assign_ids_and_edges`, `compute_annotations`, and `walk_nodes`, then contracts accepted common masks with `contract_node_to_leaf`. The output is a `ReducedInstance` containing the two reduced roots plus `atom_to_original_mask`, which is later consumed by candidate generation, packing, and output expansion.

### 7.3 Candidate Validation

Candidate generation is intentionally permissive: it proposes masks that might be useful components. The validator is the safety layer. For each candidate mask, it checks that the restricted signatures agree in both trees and records the structural resources used by the component. Invalid guesses are discarded before packing.

Keeping validation central reduces coupling. Candidate generators are allowed to be simple because correctness is not distributed among many generator-local special cases.

Validation starts with cheap exits: empty masks and singleton masks are not stored as candidates because singleton completion is handled during output assembly. Two-leaf masks use a specialized pair validator that computes the path edges and component-root masks in both trees. Three-leaf masks first pass a topology precheck, which avoids expensive restricted-signature construction for obviously incompatible triplets. General masks find the induced component root in each tree, compute the restricted signature and used component edges, and accept the mask only when the two restricted signatures are identical.

The returned `ValidationInfo` is turned into a `Candidate`. Its weight defaults to  $|A_c| - 1$ , matching the number of singleton components it can save if packed. The pool deduplicates candidates by `atom_mask` and invalidates any previously cached conflict structure when a new candidate is added.

*Main implementation components.* The validation layer is concentrated in `CandidatePool`. `validate_mask` and `CandidatePool.validate_pair_mask`. Pair masks use `pair_path_edges_and_root_mask`; triplets use `triplet_topology_precheck`; general masks use `find_component_root_by_rank` or `find_component_root` followed by `restricted_signature_and_component_edges`. Accepted masks enter the pool through `CandidatePool.add_mask_weighted`, which creates the `Candidate` from the cached `ValidationInfo`.

### 7.4 Metafeature Extraction and Use

The guarded architecture intentionally avoids a complex top-level metafeature router. The top-level decision is only the size gate  $n \leq 2200$ . However, metafeatures remain important inside the kernels, where they choose conservative packing modes, repair budgets, and small local refinements.

The main metafeatures are computed from the input or reduced trees:

- **Atom ratio** is `atom_count`/ $n$ . A low value means exact common-subtree reduction found substantial shared structure; a value near one means the instance stayed hard after safe reduction.
- **Cluster Jaccard** compares the sets of internal subtree masks in the two trees, excluding the full-tree mask:

$$\frac{|M_1 \cap M_2|}{\max(1, |M_1 \cup M_2|)}.$$

It measures how much common cluster structure remains after parsing or reduction.

- **Soft-cherry agreement** is sampled on small instances. For each cherry in one tree, the solver asks whether the corresponding two leaves have an LCA of at most eight leaves in the other tree. The two directions are averaged.
- **Triplet agreement** samples up to 900 triples and compares their rooted topology code in both trees. It is deterministic because the random sample uses a fixed seed derived from  $n$ .
- **Hard-edge bucket** is a boolean flag for small instances with high atom ratio, very low cluster Jaccard, low soft-cherry agreement, and low triplet agreement. In that region, candidate choices tend to collide on scarce edge resources, so packing becomes more edge-conservative.

These features are used locally. In the large core, `cluster_jaccard` can enable direct repair on medium-sized atom counts, while `hard_edge_bucket` switches the direct greedy packer to prefer lower edge cost when candidate weights tie. For dense-seed refinements, `atom_ratio`, `cluster_jaccard`, soft-cherry agreement, and triplet agreement decide whether alternate seeds or multistart refinement are worth the time. In the small/medium branch, closely related reduced metafeatures tune top- $k$  and round-seed parameters for medium cases. The important architectural point is that metafeatures modify bounded local behavior; they do not select from a large portfolio of unrelated solvers.

*Main implementation components.* `compute_instance_meta` creates the `InstanceMeta` record. It reuses `compute_annotations`, `walk_nodes`, and the reduced atom count, then calls `sampled_soft_cherry_lca8` and `sampled_triplet_agreement` for the small-instance signals. Local policy predicates such as `should_use_pressure_repack`, `should_refine_alternate_seed`, and `should_dense_meta_multistart` consume the resulting record inside the kernels.

## 7.5 Score-Sensitive Small/Medium Kernel

The small/medium kernel exists because the score formula makes these instances fragile. In the ablation, pure single-core performance was most damaged by small denominators rather than by huge raw component counts. The worst single-core gaps included cases where one to three extra components caused score losses above five points.

The kernel therefore spends more effort on alternatives and local refinements for instances with  $n \leq 2200$ . The cutoff is intentionally simple. It protects the region where the single core lost the most score while preserving the architectural reduction for larger instances.

Implementation-wise, this branch invokes the embedded score-sensitive solver with a bounded time budget and a small number of parameter schedules. Tiny instances receive additional trials with larger top- $k$ , round-seed, and scoring-round settings because their score denominators are small. Medium-small instances can receive one of two parameter substitutions guided by metafeatures. A low atom-ratio case with strong common-cluster signal uses a wider top- $k$  setting, while a balanced medium case uses a different top- $k$  and seed-round balance. These substitutions are based on reduced atom ratio and reduced cluster Jaccard, not on instance names.

The branch keeps the best output among its attempts by component count. For very small high-margin cases, it also invokes the high-margin split check, which accepts only validated component-count improvements. The result then passes through the unified repair acceptance stage described below. In the h05 trace, the kernel reaches 292 components and unified repair improves the incumbent to 290.

The embedded score-sensitive solver is a self-contained smaller-instance solver carried inside the final single-file submission. It is embedded as source text rather than imported as a separate module, so the Optil-facing script keeps one deployment artifact while still preserving a distinct small-instance algorithm. The wrapper executes it with redirected standard input and

output, sets a local `TIME_LIMIT`, temporarily applies parameters such as `TOPK`, `ROUND_SEEDS`, `SCORE_ROUNDS`, and `SCORE_RANDOM_INTERVAL`, and then restores the previous environment. This makes the embedded solver behave like an internal kernel call rather than a separate portfolio race.

Internally, the embedded solver uses a different search shape from the large candidate-pool core. It parses the two binary Newick trees into compact arrays of nodes with parent pointers, leaf masks, and depths. It then generates candidate component masks with a top- $k$  dynamic program over pairs of rooted subtrees. For a subproblem  $(u, v)$ , the table stores up to  $k$  large compatible masks that can be realized inside the two rooted subtrees. Internal subproblems combine the two possible child matchings and also keep suppression/skipping alternatives, which is what gives the small solver a richer set of near-best local components than the simpler large-instance core.

Those masks are converted into a compact resource model. Each candidate uses the label bits it covers plus internal-resource bits from both input trees. A selected set is feasible when these resource bitsets are disjoint, so the embedded solver is also solving the same underlying set-packing problem, but with a score-oriented candidate bank and scoring procedure. Candidate ordering comes from a lightweight score surrogate: when array operations are available, the solver may use a small primal-dual style relaxation as an ordering hint; otherwise it falls back to pure-Python scarcity and greedy-packing scores. The final forest is still chosen by deterministic rounding and disjoint-resource packing, not by trusting the fractional surrogate directly.

The embedded solver has two important alternative routes. If exact common-subtree contraction produces a useful smaller instance, it can solve a contracted route and expand the selected atom components back to original labels. If contraction is weak, it can solve directly on the original leaves, and on some medium cases it adds span-diverse candidates so that one large high-scoring mask does not crowd out several smaller but jointly better masks. The wrapper selects the lowest component count among the attempted embedded routes, then hands the resulting forest to the unified repair stage.

*Main implementation components.* The wrapper entry point is `_exec_score_sensitive_small`. It uses `_medium_small_route_env` to choose feature-based parameter substitutions, then launches bounded solver attempts through `_exec_small_capture`, which executes the `_SMALL_SOLVER` source string in a redirected stdin/stdout context. Inside that embedded solver, `generate_topk_candidates` builds the dynamic-programming candidate bank, `maybe_build_contracted_instance` and `solve_contracted_route` handle useful exact contraction, `solve_flat_route` and `solve_span_flat_route` handle original-leaf routes, `build_resource_model` creates the bit-resource packing instance, `candidate_scores` computes ordering scores, and `deterministic_round` selects a disjoint component set. The selected partition is rendered as ordinary output text before the unified repair stage receives it.

## 7.6 Single Large Candidate-Pool Core

For larger instances, the fixed architecture uses a single candidate-pool core. It constructs candidate components from the reduced trees, validates them, and packs a compatible subset. The important simplification is that there is no large portfolio of independent routes and no sequence of many late postprocessors competing for the same residual defects.

The core begins from singleton atoms, then builds a `CandidatePool`. The pool stores annotations, leaf arrays, depth maps, and leaf-order ranks for both reduced trees. In the default guarded mode the pool is often first used as a metadata and validation object rather than as a large static candidate list. Merge-repair routines then generate candidate unions from the current forest, order-adjacent components, or blocker regions, validate those unions through the pool, and accept only resource-disjoint replacements. This keeps candidate construction lazy and local while preserving a single validation and packing representation.



The candidate universe is generated in layers. The first layer contains cheap local structure: close leaf pairs, short intervals in tree order, small common chains, near-clades, and pair-cluster masks. These are useful before any incumbent is known. The second layer expands promising seeds by pair unions, which produces larger mergeable components while keeping candidate counts capped. The third layer is incumbent-driven: after the solver has a current atom partition, repair routines focus on unions of adjacent current components, common-cluster boundaries, split/repack regions, and counterfactual blocker regions. This is the layer that spends time on the residual defects that actually remain.

This layered generation is the main reason the large core can stay simple. The set of all label subsets is impossible to enumerate, and even a large static candidate pool would force expensive validation and conflict bookkeeping. The fixed core instead keeps `CandidatePool` as the shared validation and metadata object, and lets each local repair generate only the masks it needs. Every mask still passes through the central validator before it can affect the packing.

The output of the large branch is produced by improving a set of atom masks. The trace for h26 illustrates the scale: 14734 leaves are safely reduced to 12206 atoms, and set-packing/repair reduces the working partition to 6902 components. Output assembly then expands each selected atom mask back to the original labels. The branch deliberately favors a small number of transparent stages over additional late recovery machinery: exact reduction, metafeature summary, candidate packing/repair, and output assembly.

*Main implementation components.* The wrapper entry point is `_exec_single_core`, which calls the large-core `solve_instance`. Inside that core, `rounds_for_atom_count` chooses the bounded candidate schedule and `build_candidate_pool` builds the metadata-bearing `CandidatePool`. Candidate sources include `inject_proximity_candidates`, `generate_seed_masks`, `generate_pair_masks`, `generate_order_interval_masks`, `generate_common_chain_masks`, `generate_near_clade_masks`, `generate_pair_cluster_masks`, and `expand_by_pair_unions`. The primary atom-mask improvement path uses `primary_repair_atom_masks`, `repair_forest_order_atom_masks`, `progressive_extra_repair`, and `late_wide_order_repair`. Optional dense refinements use `build_small_seed_pool`, `small_anytime_refine`, `selected_feedback_refine`, and guarded local repackers such as `small_split_repack`, `edge_conflict_repack`, and `counterfactual_blocker_repack`. Final atom masks are expanded by `atom_masks_to_components` and installed with `set_current_components`.

## 7.7 Conflict-Aware Packing

Packing selects compatible candidate components. Compatibility has two dimensions: selected masks must partition labels, and their induced tree resources must not conflict. The packer uses greedy conflict-aware selection rather than constructing a heavyweight global optimization model. This keeps the implementation small enough for large instances.

Any uncovered atom is emitted as a singleton component. This is not a quality feature; it is the completion rule that turns a selected packing into a full valid partition.

There are two packing paths. If the candidate list is large or the atom count is large, the solver uses direct greedy packing without building the full  $O(|C|^2)$  conflict graph. It scans candidates in several deterministic and randomized orders. The static score is roughly

$$\frac{10000 \cdot w_c + 50 \cdot |A_c|}{\max(1, |E_c^1| + |E_c^2|)},$$

so high-weight, large, low-edge-cost candidates are preferred. Additional orders sort by raw weight, by size, or by edge-cost-per-weight. During a scan, the packer keeps three used-resource masks: atoms, tree-one edges, and tree-two edges. A candidate is selected exactly when all three intersections are empty.

If the candidate list is smaller, the solver may build conflict bitsets. Candidates conflict when atom masks overlap or either edge mask overlaps. Connected conflict components up to

a small limit are solved exactly; larger components use randomized greedy. A light local repair pass can remove and replace candidates around the current support, but it still accepts only resource-disjoint candidates and only through the same set-packing view.

*Main implementation components.* The scalable path is `solve_pool_direct_greedy`, with local exchange handled by `direct_repair_selected`. The smaller-pool path is `solve_pool`; it calls `CandidatePool.ensure_conflicts`, `candidates_conflict`, `conflict_components`, `solve_component_exact`, and `solve_component_randomized_greedy`. The selected candidate indices are normalized by `assemble_solution_indices` and converted back to atom masks with `selected_indices_to_atom_masks`.

## 7.8 Connection to Weighted Set Packing

The large branch is best understood as an approximate weighted set-packing solver over valid component candidates. After exact reduction, let  $U$  be the set of atoms. A candidate  $c$  covers a subset  $A_c \subseteq U$  and uses edge/resource sets  $E_c^1$  and  $E_c^2$  in the two restricted trees. Selecting a component of size  $|A_c|$  saves approximately  $|A_c| - 1$  components relative to singleton atoms, so its packing weight is tied to that saving. The idealized optimization problem is

$$\max \sum_{c \in C} w_c x_c$$

subject to

$$\sum_{c: a \in A_c} x_c \leq 1 \quad \forall a \in U, \quad \sum_{c: e \in E_c^1} x_c \leq 1 \quad \forall e, \quad \sum_{c: e \in E_c^2} x_c \leq 1 \quad \forall e, \quad x_c \in \{0, 1\}.$$

The atom constraints prevent two chosen components from covering the same reduced leaf atom. The edge constraints encode agreement-forest compatibility. Uncovered atoms are emitted as singleton components, so a partial packing still yields a complete valid partition.

As a concrete small packing example, let the reduced atom universe be

$$U = \{a, b, c, d\}$$

and suppose candidate validation has accepted

$$x = \{a, b\}, \quad y = \{b, c\}, \quad z = \{c, d\}.$$

Each candidate has weight 1, because it merges two singleton atoms and saves one component. The candidates  $x$  and  $y$  cannot both be selected because they overlap on atom  $b$ ; likewise  $y$  and  $z$  overlap on atom  $c$ . If  $x$  and  $z$  have disjoint edge masks in both trees, the packer can select  $\{x, z\}$ , covering all four atoms with two components. If an edge conflict blocks  $z$ , the packer may select  $x$  and leave  $c$  and  $d$  as singleton components. Both outcomes are valid partitions; the first is simply better.

The implementation does not solve this integer program with an external MIP solver. Instead, it validates candidates, represents atom and edge resources as bit masks, and applies direct greedy packing plus local repair. For smaller candidate pools, the older core can also reason through conflict components. Thus the architecture is connected to set packing at the modeling level, while the contest implementation uses direct heuristic approximations.

*Main implementation components.* The set-packing model is embodied by the `Candidate` fields `atom_mask`, `edge_mask_1`, `edge_mask_2`, and `weight`. Candidate creation is centralized in `CandidatePool.add_mask_weighted`; feasibility is checked by `candidates_conflict` or by the used-resource masks inside `solve_pool_direct_greedy`. Singleton completion and expansion are handled by `atom_masks_to_components`, `selected_masks_to_components`, and `render_output`.

## 7.9 Unified Repair and Acceptance

The small/medium branch ends with one unified repair acceptance stage. Earlier versions had multiple wrapper postprocessors: safe pair merge, high-margin split, common-cluster refinement, structured repack, and local destroy-rebuild. The fixed architecture collapses this into one acceptance surface. Internally, candidate improvements are allowed only if the resulting partition is valid and does not increase component count.

This reduces accidental coupling. The rest of the solver does not need to know which micro-repair produced an improvement. It only sees a validated incumbent and an accepted replacement.

The implementation shares one parsed and reduced repair context across the internal repair recipes. This avoids repeatedly parsing trees, recomputing annotations, recomputing exact reduction, and rebuilding candidate metadata. Each recipe receives the current incumbent and may propose a replacement. A replacement is accepted only after it is rendered, counted, and validated as a partition over the original labels. Equal or worse component counts are discarded, which makes the stage monotone with respect to the objective.

The repair stage is bounded by the same wall-clock guard as the rest of the solver. If the remaining budget is small, it returns the incumbent rather than starting a repair whose acceptance cannot be evaluated cleanly.

*Main implementation components.* The public entry point of this stage is `_unified_repair_output`. It owns the shared `_ACTIVE_REPAIR_CONTEXT`, which is created by `_build_repair_context` and accessed through `_repair_context`, `_repair_context_annotations`, and `_repair_context_small_pool`. The internal recipes are `_safe_pair_merge_output`, `_small_high_margin_split_output`, `_common_cluster_refine_output`, `_structured_repack_output`, and `_destroy_rebuild_lns_output`. Candidate replacements are rendered and checked by `_render_valid_atom_masks`, `_render_valid_original_masks`, `validate_components`, and `render_output`.

## 7.10 Output Assembly and Completion

The final stage converts the internal atom-level representation back into the forest format expected by the judge. Its first responsibility is expansion: when exact common-subtree reduction has contracted several original leaves into one atom, every selected atom mask must be decoded through `atom_to_original_mask`. Its second responsibility is completeness: any atom not covered by a selected multi-atom component is emitted as a singleton. This is why even a partial packing is still a valid full partition.

*Main implementation components.* Atom-to-label expansion is handled by `decode_original_mask`, `selected_masks_to_components`, and `atom_masks_to_components`. Forest text is produced by `component_to_newick` and `render_output`, and checked by `validate_components`. Complete singleton components are created by `singleton_components`, and the accepted component list is installed with `set_current_components`.

## 7.11 Anytime Runtime Discipline

The solver is an anytime algorithm: it maintains a complete incumbent and then improves that incumbent as time remains. This is different from a batch pipeline that only becomes complete at the end. Each stage works against the current incumbent, and only accepted improvements replace it.

Time-budget handling is distributed but simple. Long loops check the shared deadline before expanding candidates, validating masks, building conflict structures, running randomized greedy restarts, or exploring local repair regions. Wrapper stages receive bounded local budgets, and repair stages stop when the remaining time is too small to evaluate and render a proposed

replacement. This discipline is especially important on large instances: the architecture can spend time on useful local improvements without making any one optional refinement part of the correctness path.

*Main implementation components.* The shared time guards are `time_left` and `expired`. Wrapper budgets are computed by `_time_limit` and `_requested_time`. Postprocess stages refresh their deadline through `_reset_postprocess_deadline`. The embedded solver uses the same pattern through `configured_wall_limit` and `internal_time_limit`.

## 8 Motivation for the Guard

Table 2 summarizes the observed score gap that appears when the small/medium branch is removed and every instance is forced through the single candidate-pool core. The measurement uses an 80-instance ablation sample and reports score points.

| Instance bucket         | Cases | Mean score gap | Gap sum | Component gap |
|-------------------------|-------|----------------|---------|---------------|
| $n \leq 200$            | 28    | 4.173          | 116.841 | 36            |
| $201 \leq n \leq 700$   | 20    | 4.400          | 88.009  | 157           |
| $701 \leq n \leq 2200$  | 16    | 3.864          | 61.822  | 299           |
| $2201 \leq n \leq 6000$ | 8     | 2.017          | 16.138  | 215           |
| $n \geq 6001$           | 8     | 0.843          | 6.741   | 511           |

Table 2: The score gap is largest in small and medium regions, even though the largest raw component gap appears on huge instances. This is why the guarded architecture restores the small/medium kernel but keeps the large branch simple.

The table motivates the guard directly. Adding more large-instance machinery would address the smaller part of the score gap. Restoring the score-sensitive kernel for  $n \leq 2200$  recovers most of the lost score with only one top-level rule.

## 9 Evaluation

The quick proxy was designed to avoid repeated 100-instance sweeps. It contains small score-sensitive sentinels, one medium tuning case, one huge bottleneck, and two large pressure cases. A second proxy swaps in h41 to check another large structure. Both proxies are conservative relative to the known full score.

### 9.1 Stage Instrumentation

The solver can emit JSONL stage traces by setting `PACE26_STAGE_LOG`. For the guarded architecture, the trace includes the policy gate, branch choice, major kernel calls, component-count changes, and elapsed time. For the single-core large branch it also records exact common-subtree reduction, metafeature computation, set-packing/repair, output assembly, and total time. This instrumentation is passive: if the environment variable is unset, the solver does not write trace files.

*Main implementation components.* Wrapper-stage events are emitted by `_stage_run_text` and `_stage_instant`, both of which call the shared `_stage_event` logger when `PACE26_STAGE_LOG` is set. `_stage_run_text` also calls `_output_count` before and after a stage so the trace records component-count movement, not only elapsed time. The large core emits its own events from `solve_instance` for exact reduction, metafeature summary, candidate-pool construction, set-packing/repair, output assembly, and total runtime.

| Instance | Stage                                     | Before | After | Seconds |
|----------|-------------------------------------------|--------|-------|---------|
| h05      | guarded policy gate:<br>small/medium      | –      | –     | 0.000   |
| h05      | score-sensitive small/medium<br>kernel    | –      | 292   | 29.614  |
| h05      | unified repair acceptance                 | 292    | 290   | 7.885   |
| h05      | guarded architecture total                | –      | 290   | 37.502  |
| h26      | guarded policy gate: large<br>single core | –      | –     | 0.000   |
| h26      | exact common-subtree<br>reduction         | 14734  | 12206 | 1.826   |
| h26      | metafeature summary                       | –      | –     | 0.302   |
| h26      | weighted set-packing and<br>repair        | 12206  | 6902  | 63.096  |
| h26      | output assembly                           | 6902   | 6902  | 0.045   |
| h26      | guarded architecture total                | 14734  | 6902  | 67.173  |

Table 3: Representative stage traces produced by the instrumented guarded-core solver. Counts are component or atom counts as appropriate for the stage.

## 9.2 Paired Statistical Comparison

The solver contains randomized and greedy choices, including the random seed, greedy restart schedules, candidate limits, repair passes, and time-dependent stopping behavior. Therefore the relevant experimental unit is not an unpaired score in isolation, but a paired result on the same benchmark instance under the same time budget. In the seed-aware comparison below, each block is an (instance, seed) pair. The cost of a configuration is the best component count, equivalently the best score, obtained within the fixed computation time.

The comparison is an ablation of the guard rather than a comparison between two unrelated solvers. The *previous single-core* variant uses the same tree representation, exact reduction, candidate validator, and weighted set-packing interpretation, but removes the top-level size guard and sends every instance through the general candidate-pool core. The *guarded-core* variant restores the single rule from Section 7: score-sensitive small/medium instances use the specialized kernel and repair acceptance stage, while larger instances continue to use the same single candidate-pool core. The seven proxy instances are run under both variants with the same two seeds, so the resulting 14 paired blocks isolate the effect of the guard and its small/medium kernel rather than differences in parsing, validation, output format, seed selection, or benchmark selection.

For two architectures, the primary test is the Wilcoxon signed-rank test on paired score differences. A sign test is also reported because it uses only win/loss/tie information. If three or more configurations are compared, the same blocking idea extends to a Friedman test over instances or instance-seed blocks, followed by post-hoc comparisons against the best configuration. This non-parametric paired analysis is more appropriate here than a plain one-way ANOVA, because both architectures are evaluated on the same instances.

The guarded architecture wins every non-tied block and loses none. The practical effect is also visible: it removes 107 components over the 14 paired blocks, with a mean score gain of 3.527 points. The runtime increase is expected because the guarded branch deliberately spends more time on score-sensitive small and medium instances, but all runs remain within the per-instance time limit.

The projected full public-set score of the guarded architecture is 97.356217. The projection

| Quantity                             | Value                             |
|--------------------------------------|-----------------------------------|
| Architectures compared               | pure single core vs guarded core  |
| Instances                            | h00, h05, h08, h25, h26, h43, h88 |
| Seeds                                | 20260501, 20260502                |
| Paired blocks                        | 14                                |
| Win/tie/loss blocks for guarded core | 8 / 6 / 0                         |
| Mean score, previous architecture    | 92.731                            |
| Mean score, guarded architecture     | 96.258                            |
| Mean paired score difference         | +3.527                            |
| Median paired score difference       | +0.768                            |
| Total component-count difference     | -107                              |
| Mean runtime difference              | +39.571s                          |
| Wilcoxon signed-rank test            | $p = 0.0078125$                   |
| Sign test                            | $p = 0.0078125$                   |

Table 4: Paired statistical comparison on the score-proxy instances. Positive score differences and negative component-count differences favor the guarded architecture. Ties are retained in the practical win/tie/loss count and omitted from the signed-rank statistic.

| Architecture /<br>preset         | Instances                            | Mean score | Worst score | Wall time |
|----------------------------------|--------------------------------------|------------|-------------|-----------|
| Pure single core,<br>score proxy | h00, h05, h08, h25, h26, h43,<br>h88 | 92.848     | –           | 126.7s    |
| Guarded core,<br>score proxy     | h00, h05, h08, h25, h26, h43,<br>h88 | 96.258     | 92.303      | 441.9s    |
| Guarded core,<br>h41 proxy       | h00, h05, h08, h25, h41, h43,<br>h88 | 95.992     | 92.303      | 347.4s    |

Table 5: Quick-proxy results for the simplified fixed architecture.

combines measured guarded-architecture rows with measured rows for the same small/medium kernel used by the guarded branch; no unmeasured substitute rows are used. The result is comfortably above the 95-score target while keeping the branch structure compact.

## 10 Complexity Control

The main complexity reduction is architectural rather than cosmetic. The fixed design removes three failure modes that made earlier attempts hard to reason about.

1. **No late-stage portfolio.** The simplified large branch does not run many independent postprocessors that fight over the same time budget.
2. **One visible policy gate.** The branch decision is based on instance size, not on a fragile conjunction of many metafeatures.
3. **One acceptance boundary.** Repair improvements are accepted only through a single validated incumbent interface.

The design still keeps the necessary complexity: tree annotations, exact reduction, candidate validation, packing, and singleton completion. These are not optional ornamentation; they are the layers that make the output valid and the search scalable.

## 11 Reproducibility and Input Use

The solver mode described here is activated by setting `PACE26_SIMPLE_GUARDED_CORE=1`. The cutoff can be adjusted with `PACE26_SIMPLE_GUARDED_SMALL_MAX_N`, whose default is 2200. The solver reads only the input instance and environment parameters. It does not read cached solution files, public output maps, instance-name tables, CP-SAT models, or external MIP solver state.

## 12 Conclusion

The key observation behind the final design is that the pure single-core collapse removed the wrong thing: small/medium score sensitivity was more important than large raw component count. Restoring that one solver family behind a single size gate recovers most of the score while keeping the large-instance architecture simple. The result is a solver architecture that is easier to explain, easier to test, and still comfortably above the 95-score target in the quick and projected evaluations.